

Hosting & Budget Plan

Netlify-led deployment proposal | 19 September 2026 | Version 1.0

Decision context. The project owner has selected Netlify for the frontend because it is already used for other projects. This document translates that choice into a proposed deployment architecture, operating budget and release plan.

Recommended launch approach

Use Netlify for the website; Railway for the API, background processing, Redis and LiteLLM gateway; Supabase for PostgreSQL with pgvector and file storage; and Firebase for authentication. Model inference is billed separately by the selected AI provider.

Stage	Planning allowance / month	Basis
Development pilot	\$150	Up to \$65 hosting + \$50 AI; remainder is contingency.
Small commercial launch	\$250	\$70-135 hosting + \$50 AI at the economical example rate; 25% contingency fits within \$250.
Higher-traffic scenario	\$2,300-3,500	\$1,805-2,760 hosting plus AI estimate, rounded upward with about 25% contingency.

These allowances assume controlled AI usage. The higher-cost AI example raises the growth budget to approximately \$19,200-20,400 including 25% contingency. Neither example is a provider quotation or a guaranteed spending cap.

What stakeholders are being asked to endorse

Endorse the selected architecture and an initial pilot allowance; nominate billing and technical owners; and require a pilot cost review before approving public launch or a larger traffic commitment.

Status and scope

This is a planning document. No hosting resources have been provisioned, no deployment has been performed, and no spending approval is implied. Repository documentation identifies the application stack; production readiness has not been independently certified.

All amounts are USD per month before tax. Engineering effort, domain purchase, transactional email, optional paid monitoring, enterprise support and separate always-on staging environments are excluded. Finance should apply its approved exchange rate and tax treatment for local-currency budgeting.

Architecture & Ownership

Retain existing integrations and place long-running application services on Railway.

Component	Selected platform	Role / responsibility
Frontend	Netlify	Next.js website, deployment previews and frontend delivery. Validate framework compatibility during staging. [1, 2]
API and workers	Railway	NestJS API and background processing. Engineering owns builds, sizing, health checks and releases.
Queue / cache	Railway Redis	Shared cache and background queues; validate persistence, eviction and retry behavior.
Database	Supabase	PostgreSQL with pgvector for application records and semantic retrieval. Engineering owns schema, permissions and query tuning.
File storage	Supabase Storage	Private documents and generated outputs. Use authenticated access or short-lived signed URLs.
Authentication	Firebase	Existing email/social sign-in; API validates identity tokens and organization access.
AI gateway	Railway / LiteLLM	Gateway routes requests to the chosen provider; enforce model selection and request budgets.
AI inference	Model provider	Token-based or provider-specific usage fees; separate from gateway hosting.

Request and document flow

User opens Netlify website and signs in through Firebase. The browser sends an authenticated request to the Railway API. The API authorizes access, queries Supabase and sends model requests through LiteLLM. Redis coordinates queued work; workers process documents and store results in Supabase.

Accountable owners

Product sponsor: budget and launch scope. **Technical lead:** deployment, credentials, tenant isolation, restore testing and incident response. **Finance / account owner:** invoices, shared Netlify allocation and usage alerts. Named individuals remain to be assigned.

Keep Railway and Supabase geographically close where available. Choose the region based on users and data requirements. Multiple vendors introduce network traffic charges and dependencies; the application team owns end-to-end reliability.

Monthly Hosting Budget

Planning estimates for one environment; AI inference is excluded on this page.

Assumption	Development	Small launch	Higher traffic
Active users / month	5-20	100-500	10,000
AI interactions / month	1,000	10,000	300,000
Stored files	1 GB	10 GB	100 GB
Outbound data / month	10 GB	100 GB	1 TB

Service	Development	Small launch	Higher traffic
Netlify allocation	\$0-25	\$20-40	\$40-200
Railway API / workers	\$10-25	\$15-40	\$150-500
Railway Redis	\$3-5	\$5-10	\$20-80
Railway LiteLLM	\$5-10	\$5-15	\$20-80
Supabase database	\$0	\$25	\$75-300
Supabase storage overage	\$0	\$0-5	\$0-100
Firebase Auth*	\$0	\$0	\$0
Hosting total	\$18-65	\$70-135	\$305-1,260

Model assumptions: light text/document workloads, no GPU hosting, no multi-region redundancy and a tuned database. Growth estimate assumes traffic is split across frontend, API and direct file downloads. It is not a quote for 1 TB on every provider. Peak concurrency, token lengths and vector-index size remain unmeasured.

How to interpret the bill

Railway Hobby has a \$5 usage minimum; Pro has a \$20 minimum, both with included usage credit. The three Railway rows are allocations within one bill, not three plan subscriptions. Database and storage rows avoid charging Supabase plan fees twice. [3, 4]

*Firebase assumes ordinary email/social authentication within applicable quotas, excluding SMS and enterprise identity options. Free database allowances are appropriate for a limited pilot, not the production budget baseline. [4, 5]

Usage & Cost Sensitivity

AI usage is the largest uncertain cost. Shared Netlify usage also requires account-level review.

Netlify: existing account versus new cost

Published credit-based plans are Free (\$0), Personal (\$9) and Pro (from \$20). The existing account may use different or legacy terms. Confirm its plan and spare credits before attributing incremental cost to this project. [1]

For current credit pricing: production deploys use 15 credits each, bandwidth 20/GB, compute 10/GB-hour and requests 2 per 10,000. Pro starts with 3,000 credits; additional credits are \$10 per 1,500 when recharge is enabled. [1]

Example: 100 GB frontend delivery + 20 production deploys + 1 million requests + 10 GB-hours compute = 2,600 credits. Other projects consume the same account allowance. Avoid double-counting the existing subscription; allocate its cost internally and budget actual overage separately.

AI sensitivity, not model pricing

Monthly interactions	At \$0.005 each	At \$0.05 each
1,000	\$5	\$50
10,000	\$50	\$500
300,000	\$1,500	\$15,000

These are hypothetical average costs per completed interaction, not current prices for any named model. Include all model calls, retries, retrieved context and output tokens when measuring this average. Embeddings, tool fees, OCR, image/audio processing and premium features require additional allowances.

Hosting + AI, before contingency	Economical example	Higher-cost example
Development	\$23-70	\$68-115
Small launch	\$120-185	\$570-635
Higher traffic	\$1,805-2,760	\$15,305-16,260

Budget control

Measure 100 representative completed interactions during the pilot. Apply per-user quotas, token ceilings and bounded agent steps. Configure billing alerts and an application-enforced stop or approval rule at the agreed AI allowance; alerts alone do not cap spend. Review weekly during launch and monthly after stabilization.

Consolidation Options

Fewer vendors can simplify billing, but migration and operations also have a cost.

Option	Dev / growth hosting	Implications
Selected stack Netlify + Railway + Supabase + Firebase	\$18-65 / \$305-1,260	Preserves the frontend decision and existing managed integrations. Recommended launch baseline.
Mostly Railway	\$25-65 / \$300-1,200	Move web, database and storage to Railway. Firebase and model provider remain external. Requires changing the Netlify decision.
Railway + self-hosted auth	\$35-90 / \$350-1,500	App services consolidated; replace Firebase and operate authentication. Model API remains external.
AWS managed services	\$100-250 / \$500-2,000	ECS/Fargate, RDS, ElastiCache, S3 and Cognito. Bedrock can consolidate supported model billing. Significant integration and setup work.
Single VPS with Docker	\$20-60 / \$150-700+	Operate apps, database, Redis and auth yourself. External model API. A single server is not equivalent to a redundant managed deployment.

Alternative figures are broad planning allowances for the same usage scenarios, excluding AI, migration labor and enterprise features. They are not validated capacity estimates. AWS and VPS costs depend strongly on region, sizing, backup and availability choices. [6-9]

Why retain the selected architecture

The owner already uses Netlify. Keeping Firebase and Supabase avoids introducing authentication migration and database operations into the initial publishing scope. Railway runs the persistent API and worker processes without forcing them into frontend functions.

Railway offers S3-compatible buckets, but its database templates are unmanaged: the application team remains responsible for database maintenance and recovery. Vendor consolidation therefore does not automatically reduce operating effort. [6, 7]

When to revisit

Reassess after measured usage shows material savings, a customer requires a specific cloud or region, or the team can support the operational burden. Do not migrate solely to reduce the number of invoices.

Release Plan & References

Launch through a measured pilot, with clear ownership and a recoverable release.

Gate	Required result	Owner
1. Accounts and budget	Confirm Netlify plan, region, billing owner, AI provider and pilot allowance.	Sponsor / finance
2. Deployment preparation	Validate Next.js on Netlify; production API/worker builds, health checks, environment settings and database migrations.	Engineering
3. Staging verification	Test login, tenant access, chat streaming, uploads, vector retrieval, queued jobs and error handling. Keep backend secrets out of browser builds.	Engineering / QA
4. Recovery and cost test	Verify database/file recovery and rollback. Measure AI unit cost; load-test expected concurrency and set quotas.	Technical lead
5. Pilot then public launch	Release to limited users; review errors, latency and spend. Sponsor approves public rollout against measured results.	Product / engineering

No delivery dates are committed in this document. Scope and timing depend on deployment readiness, account access and staging results. Production database changes should use reviewed migrations and a recovery plan.

Sources and evidence

Official sources checked 19 September 2026. Published prices may change; actual account contracts and invoices take precedence. Bracketed references support platform facts, not the workload estimates.

[1] [Netlify pricing](#)

[2] [Next.js on Netlify](#)

[3] [Railway pricing](#)

[4] [Supabase pricing](#)

[5] [Firebase pricing](#)

[6] [Railway database responsibilities](#)

[7] [Railway storage buckets](#)

[8] [AWS Fargate pricing](#)

[9] [AWS Cognito pricing](#)

Internal context: repository README and the project owner's platform selection. This brief supersedes the earlier Vercel-based frontend budget. Supabase Storage is the proposed storage choice for this plan.